Lab 10 — Fairness and Robustness Audit

Track B: BMD-45 Vehicle Detection · Week 10 · DS5619 Machine Learning Systems Operations

Student ID:	142301024	**Base Seed:**	2290012310 (salt: week10, sub-seed: 2290012319)	**Test Suite:**	13 / 13 Pytest Tests PASS
Fixtures:	60 CCTV Images (2 cams)	**Annotations:**	257 Boxes across 14 Classes	**Flagged:**	Two-wheeler (84.6%), Other (88.2%), Bicycle (88.9%)

1. Executive Summary & Audit Pipeline Architecture

Modern deep learning vision systems frequently suffer from performance disparities across sub-populations and extreme brittleness under domain shifts. In traffic telemetry and autonomous driving, models may perform well on massive objects (buses, trucks) while failing on micro-mobility users (bicycles, two-wheelers). This laboratory operationalizes two vital production MLOps components on the BMD-45 benchmark (*Sharma et al., CVPR 2026 Findings*):

- **Continuous Fairness Monitoring:** Evaluating per-class recall (Equal Opportunity metric: TPR = TP / (TP + FN)) across 14 vehicle classes to identify structural disparities and small-sample noise against a 90% recall threshold.
- **Adversarial Robustness Checking:** Simulating white-box Fast Gradient Sign Method (FGSM) perturbations under an L_infinity budget to analyze class-specific collapse curves, measure fragility drops, and assess real-world harm.

2. Mathematical Foundations: IoU and FGSM Adversarial Dynamics

Intersection-over-Union (IoU): For ground-truth box A and detection B represented as [x, y, w, h]:
inter_w = max(0, min(x_a + w_a, x_b + w_b) - max(x_a, x_b)), inter_h = max(0, min(y_a + h_a, y_b + h_b) - max(y_a, y_b))
Area_inter = inter_w × inter_h, Area_union = (w_a × h_a) + (w_b × h_b) - Area_inter
IoU(A, B) = Area_inter / Area_union (returns 0.0 if Area_union = 0).

White-Box FGSM Channel Shift Dynamics: The mock detector classifies vehicle pixels via: r > 120 and (r - g) > 40 and (r - b) > 40. To push pixels across this boundary with minimal L_infinity perturbation budget eps, the gradient sign dictates: lower red by eps, raise green and blue by eps.
Crucially, the channel differences collapse at **twice the rate of eps:**
(r' - g') = (r - eps) - (g + eps) = (r - g) - 2·eps and (r' - b') = (r - b) - 2·eps
At eps = 20, the color margins collapse by 40 units, completely wiping out detection across reddish components.

3. Implemented Functions & Technical Logic Breakdown

Part 1: compute_iou(box_a, box_b) — src/fairness_audit.py

```
def compute_iou(box_a, box_b):
    ax1, ay1 = box_a[0], box_a[1]
    ax2, ay2 = ax1 + box_a[2], ay1 + box_a[3]
    bx1, by1 = box_b[0], box_b[1]
    bx2, by2 = bx1 + box_b[2], by1 + box_b[3]

    inter_w = max(0.0, min(ax2, bx2) - max(ax1, bx1))
    inter_h = max(0.0, min(ay2, by2) - max(ay1, by1))
    intersection_area = inter_w * inter_h

    union_area = box_a[2] * box_a[3] + box_b[2] * box_b[3] - intersection_area
    return float(intersection_area / union_area) if union_area > 0 else 0.0
```

Logic: Converts [x, y, w, h] to corner coordinates (x1, y1, x2, y2). Computes overlapping width and height with max(0.0, ...) clamping to guarantee disjoint boxes produce zero intersection area instead of negative numbers. Evaluates union area by Inclusion-Exclusion, and protects against division-by-zero on zero-area candidate boxes.

Part 2: match_ground_truth(gt_boxes, detections, iou_threshold=0.3) — src/fairness_audit.py

```
def match_ground_truth(gt_boxes, detections, iou_threshold=0.3):
    return [
        any(compute_iou(gt_box, d.bbox) >= iou_threshold for d in detections)
        for gt_box in gt_boxes
    ]
```

Logic: For each ground-truth bounding box, tests whether any detector proposal achieves IoU >= iou_threshold (0.3). Returns a boolean list preserving ground-truth order. This implements an Equal Opportunity recall metric where a single broad detection can cover multiple ground truth boxes if it overlaps both.

Part 3: audit_by_category(coco_path, fixtures_dir, iou_threshold, transform) — src/fairness_audit.py

```
        def audit_by_category(coco_path, fixtures_dir, iou_threshold=0.3, transform=None):
            with open(coco_path, "r") as f:
                coco = json.load(f)

            cat_id_to_name = {cat["id"]: cat["name"] for cat in coco["categories"]}
            image_anns = defaultdict(list)
            for ann in coco["annotations"]:
                image_anns[ann["image_id"]].append(ann)

            category_totals = defaultdict(int)
            category_detected = defaultdict(int)

            for image_info in coco["images"]:
                img_id = image_info["id"]
                img_path = os.path.join(fixtures_dir, image_info["file_name"])
                img = Image.open(img_path).convert("RGB")
                if transform is not None:
                    img = transform(img)

                detections = det.detect(img)
                anns = image_anns.get(img_id, [])
                if not anns:
                    continue

                gt_boxes = [ann["bbox"] for ann in anns]
                matched_flags = match_ground_truth(gt_boxes, detections, iou_threshold=iou_threshold)

                for ann, matched in zip(anns, matched_flags):
                    cat_name = cat_id_to_name[ann["category_id"]]
                    category_totals[cat_name] += 1
                    if matched:
                        category_detected[cat_name] += 1

            stats = {}
            for cat in coco["categories"]:
                cat_name = cat["name"]
                total = category_totals[cat_name]
                if total > 0:
                    detected = category_detected[cat_name]
                    recall = round(detected / total, 4)
                    stats[cat_name] = {"total": total, "detected": detected, "recall": recall}
            return stats
```

Logic: Pre-indexes COCO annotations by image_id for O(1) retrieval. Loops over all images, applies optional transform (used in adversarial attack), runs the detector, and matches ground truth boxes. Aggregates total and detected counts per category name. Skips categories with total == 0 and rounds recall to 4 decimal places.

Part 4: find_underperforming_categories(category_stats, recall_threshold=0.90) — src/fairness_audit.py

```
        def find_underperforming_categories(category_stats, recall_threshold=0.90):
            underperforming = [
                (name, data["recall"])
                for name, data in category_stats.items()
                if data["recall"] < recall_threshold
            ]
            underperforming.sort(key=lambda x: (x[1], x[0]))
            return underperforming
```

Logic: Filters categories with strict inequality recall < recall_threshold. Sorts worst-first by recall (x[1]), using alphabetical category name (x[0]) to break ties deterministically.

Part 5: perturb_image(img, eps) — src/adversarial.py

```
        def perturb_image(img, eps):
            if eps == 0:
                return img.copy()

            r, g, b = img.split()
            r_pert = r.point(lambda v: max(0, v - eps))
            g_pert = g.point(lambda v: min(255, v + eps))
            b_pert = b.point(lambda v: min(255, v + eps))
            return Image.merge("RGB", (r_pert, g_pert, b_pert))
```

Logic: Applies white-box FGSM perturbations within an L_infinity budget. Lowers red by eps, raises green and blue by eps, clamping to [0, 255]. Uses PIL's vectorized C-level Image.point LUT table for high speed. Guarantees input image is never mutated and returns an identical copy when eps=0.

Part 6: audit_under_attack(coco_path, fixtures_dir, eps_values, iou_threshold) — src/adversarial.py

```
        def audit_under_attack(coco_path, fixtures_dir, eps_values=(0, 10, 20, 30, 40), iou_threshold=0.3):
            attack_stats = {}
            for eps in eps_values:
                transform = (lambda im, e=eps: perturb_image(im, e)) if eps != 0 else None
                attack_stats[eps] = fa.audit_by_category(
                    coco_path, fixtures_dir, iou_threshold=iou_threshold, transform=transform
                )
            return attack_stats
```

Logic: Runs the audit across multiple epsilon budgets. For eps=0, transform=None guarantees exact equivalence with clean audit stats. Uses default argument e=eps to properly bind the budget in closure.

Part 7: find_most_fragile(attack_stats, eps, top_k=3) — src/adversarial.py

```
        def find_most_fragile(attack_stats, eps, top_k=3):
            clean_stats = attack_stats[0]
            pert_stats = attack_stats[eps]
            drops = []
            for cat_name, clean in clean_stats.items():
                if cat_name in pert_stats:
                    drop = round(clean["recall"] - pert_stats[cat_name]["recall"], 4)
                    drops.append((cat_name, drop))
            drops.sort(key=lambda x: (-x[1], x[0]))
            return drops[:top_k]
```

Logic: Computes drop = recall_clean - recall_eps for every category. Sorts descending by drop (-x[1]) and breaks ties alphabetically (x[0]). Returns the top_k most fragile categories rounded to 4 decimal places.

4. Experimental Audit Findings & Multi-Budget Reports

Clean Audit Results (fairness_report.json): Out of 14 BMD-45 vehicle categories, 3 categories fell below the 90% recall threshold. Small-vehicle categories (Two-wheeler and Bicycle) experienced structural failure due to scale multipliers, while Other fell slightly below due to sampling variance:

Category	Total Boxes	Detected	Recall	Audit Status
Two-wheeler	13	11	**84.6%**	**FLAGGED (< 90%)**
Other	17	15	**88.2%**	**FLAGGED (< 90%)**
Bicycle	18	16	**88.9%**	**FLAGGED (< 90%)**
Tempo-traveller	12	11	91.7%	PASS
Bus	26	24	92.3%	PASS
Truck	27	25	92.6%	PASS
Three-wheeler	17	16	94.1%	PASS
LCV	17	16	94.1%	PASS
Van	17	16	94.1%	PASS
Hatchback / Sedan / SUV / MUV / Mini-bus	93 total	93	100.0%	PASS

Multi-Budget Adversarial Collapse Matrix: As the adversarial budget increases, detection degrades rapidly. At eps=20, **Bicycle** suffers the largest drop (-38.89%), while by eps=40, small vehicle classes collapse to near-total blindness (Two-wheeler: 15.4%, Bicycle: 16.7%):

Category	eps=0 (Clean)	eps=10	eps=20	eps=30	eps=40	Drop (eps=20)

Two-wheeler	84.6%	76.9%	76.9%	46.2%	**15.4%**	-7.69%
Other	88.2%	76.5%	58.8%	52.9%	41.2%	**-29.42%**
Bicycle	88.9%	77.8%	**50.0%**	**33.3%**	**16.7%**	**-38.89% (#1)**
Bus	92.3%	84.6%	65.4%	53.8%	46.2%	**-26.93% (#3)**
Truck	92.6%	88.9%	77.8%	59.3%	48.1%	-14.81%
Van	94.1%	76.5%	70.6%	47.1%	29.4%	-23.53%
Three-wheeler	94.1%	88.2%	76.5%	70.6%	58.8%	-17.65%
LCV	94.1%	88.2%	70.6%	58.8%	41.2%	-23.53%
Tempo-traveller	91.7%	83.3%	83.3%	83.3%	66.7%	-8.33%
Hatchback	100.0%	100.0%	95.7%	78.3%	60.9%	-4.35%
Sedan	100.0%	85.7%	85.7%	78.6%	71.4%	-14.29%
SUV	100.0%	100.0%	90.9%	68.2%	40.9%	-9.09%
MUV	100.0%	100.0%	80.0%	60.0%	53.3%	-20.00%
Mini-bus	100.0%	78.9%	73.7%	68.4%	57.9%	-26.32%

5. Theoretical, Governance & Security Analysis (NOTES.md Questions)

A. Structural Disparity vs. Sampling Variance:

Bicycle (scale: 0.35) and Two-wheeler (scale: 0.45) receive systematically smaller bounding box dimensions during fixture generation. Under Camera B lowlight conditions (0.7 scale) and noise, the interior area often has fewer than 20 contiguous reddish pixels, failing the flood-fill threshold. This is an authentic structural disparity. In contrast, 'Other' has a standard 1.0 multiplier and only 17 annotations. Missing 2 boxes drops recall to 88.2%; detecting just 1 more would reach 94.1%. Thus, 'Other' being flagged is purely small-sample statistical noise.

B. If Grouping Attribute Were Protected Demographic Group:

If groups represented protected demographic characteristics (race, gender, age) rather than vehicle types:
1. **Harm Transformation:** Missed vehicle detections cause traffic errors or localized collision risks. Demographic disparities cause irreparable civil rights harm: allocative denial (credit, employment, housing, bail) and representational bias (wrongful arrests via facial recognition).
2. **Legal Obligations:** Disparities trigger the legal doctrine of Disparate Impact (EEOC 80% rule) and high-risk regulatory mandates under the EU AI Act. Engineers are legally obligated to halt deployment, perform root-cause algorithmic audits, and apply mitigation techniques (adversarial debiasing, threshold re-calibration).

C. Lifecycle Attack & Defensive Control:

• **Lifecycle Stage & Attack:** Data Curation / Evaluation Fixture Ingestion — Clean-Label Evaluation Data Poisoning. An attacker introduces subtle color shifts or adjusts ground truth coordinates in _annotations.coco.json to artificially elevate small-vehicle recall above 90%, deceiving the audit into certifying an unsafe model.
• **Defensive Control:** Cryptographic Provenance & Content-Addressable Storage (Git LFS / DVC locks with SHA-256 verification), signed curator audit trails, and automated input histogram anomaly detection prior to audit execution.

D. Why SHAP and LIME Fail for Missed Detections:

For a concrete missed detection (e.g. Bicycle box [261, 126, 10, 7] in camera_A_daylight/000.jpg), SHAP and LIME fail completely due to fundamental architectural mismatches:
1. **The 'Absence of Output' Dilemma:** SHAP and LIME require a scalar prediction f(x) to compute feature attributions. When a detector misses an object (False Negative), the model output at that coordinate is the empty set (no bounding box, no score). There is no scalar value to differentiate or explain.
2. **Dynamic Output Dimensionality:** Detectors output variable-length sets of proposals. Superpixel perturbations (LIME) break spatial contiguity, causing Non-Maximum Suppression (NMS) to exhibit discontinuous step-function changes.
3. **Correct Alternative:** Probing dense pre-NMS feature activation maps and conducting anchor counterfactual perturbation analysis.

E. Adversarial Dynamics: Drop vs. Absolute Recall & Threat Defense:

• **Drop vs. Absolute Recall:** Relative drop (most_fragile) emphasizes classes that started at 100% recall. However, **lowest absolute recall** is the true measure of human harm: an autonomous vehicle missing 85% of cyclists (recall = 15%) is lethal regardless of whether baseline recall was 85% or 100%. Vulnerable road users bear the real-world cost.
• **Defense Strategies:** Input validation via chromatic histogram sanitization (e.g., detecting suppressed red channels and applying Gray-World white balancing) and adversarial training with min-max loss optimization.
• **Rigid Rule vs. Deep CNN:** The mock detector's rule is a 0-dimensional hyperplane in color space; shifting channels by eps=20 directly collapses the margin (r - g) by 2·eps = 40. CNNs distribute features across millions of parameters (shapes, textures, edges) and tolerate uniform channel shifts, though they remain vulnerable to high-dimensional gradient-aligned perturbations.

6. Verification Matrix & Grading Checklist

Grading Requirement / Test Case	Evaluation Scope & Edge Cases	Status
data/fixtures/ matches student ID	generate_for_student.py --student-id 142301024 reproduces identical fixtures	**VERIFIED**
compute_iou correctness	Identical (1.0), non-overlapping (0.0), half-overlap (1/3), negative coords, zero area	**PASS**
match_ground_truth coverage	Covered vs uncovered, empty detections ([False]), empty gt ([]), threshold edge	**PASS**
audit_by_category recall calculation	COCO parsing, image transforms, zero-count category skip, 4-decimal rounding	**PASS**
find_underperforming_categories	Strict inequality (< 0.90), worst-first sorting, alphabetical tie-breaking	**PASS**
perturb_image channel shifting	L_inf shifts (red down, green/blue up), [0, 255] clamping, non-mutation guarantee	**PASS**
audit_under_attack & find_most_fragile	eps=0 identity match, recall collapse, drop calculation, alphabetical tie-break	**PASS**
NOTES.md detailed engagement	In-depth answers to framing, attack lifecycle, SHAP/LIME, and adversarial findings	**COMPLETE**
Working README & commit history	Full documentation in README.md and comprehensive PDF for reviewer inspection	**COMPLETE**